

Tutorial: Single Cycle and Multi Cycle Processor Problems

Detailed Solutions with Complete Explanations

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Problem 1

We consider two CPU implementations for executing a program: **Single-Cycle (SC)** and **Multi-Cycle (MC)**.

A. Single-Cycle Processor

In the single-cycle implementation, each instruction completes in **one clock cycle**. The processor runs at a clock frequency of **80 MHz**.

If a program executes a total of **90 instructions**, calculate the **total execution time** in **nanoseconds (ns)**.

B. Multi-Cycle Processor

In the multi-cycle implementation, the clock cycle is divided into **5 shorter cycles**.

The program has the following instruction mix:

- 45 arithmetic instructions (each takes 4 cycles)
- 20 load instructions (each takes 5 cycles)
- 15 branch instructions (each takes 3 cycles)
- 10 store instructions (each takes 4 cycles)

1. Calculate the **total number of clock cycles** required to execute the program.
2. Assuming the clock frequency is **400 MHz**, compute the **total execution time** in **nanoseconds (ns)**.

C. Performance Comparison

Calculate the **speedup of the multi-cycle implementation** compared to the single-cycle processor.

Solution

A. Single-Cycle Processor

Clock frequency = 80 MHz

Each instruction takes one cycle. CPI = 1.

$$\begin{aligned}\text{Execution Time/CPU Time} &= \frac{\text{IC} * \text{CPI}}{\text{Clock Rate}} \\ \text{Execution Time} &= \frac{90 * 1}{80 * 10^6} \text{seconds} \\ &= 1125 \text{ns}\end{aligned}$$

Execution Time _{SC} = 1125 ns
--

B. Multi-Cycle Processor

$$\begin{aligned}\text{CPU Time} &= \frac{\sum_{i=1}^n (\text{IC}_i \times \text{CPI}_i)}{\text{Clock Rate}} \\ &= \frac{45 \times 4 + 20 \times 5 + 15 \times 3 + 10 \times 4}{400 \times 10^6} \\ &= \frac{180 + 100 + 45 + 40}{4 \times 10^8} \\ &= 912.5 \text{ns}\end{aligned}$$

Execution Time _{MC} = 912.5 ns

C. Performance Comparison

Speedup is defined as

$$\begin{aligned}\text{Speedup} &= \frac{\text{Execution Time}_{SC}}{\text{Execution Time}_{MC}} \\ &= \frac{1125}{912.5} \\ &= 1.23\end{aligned}$$

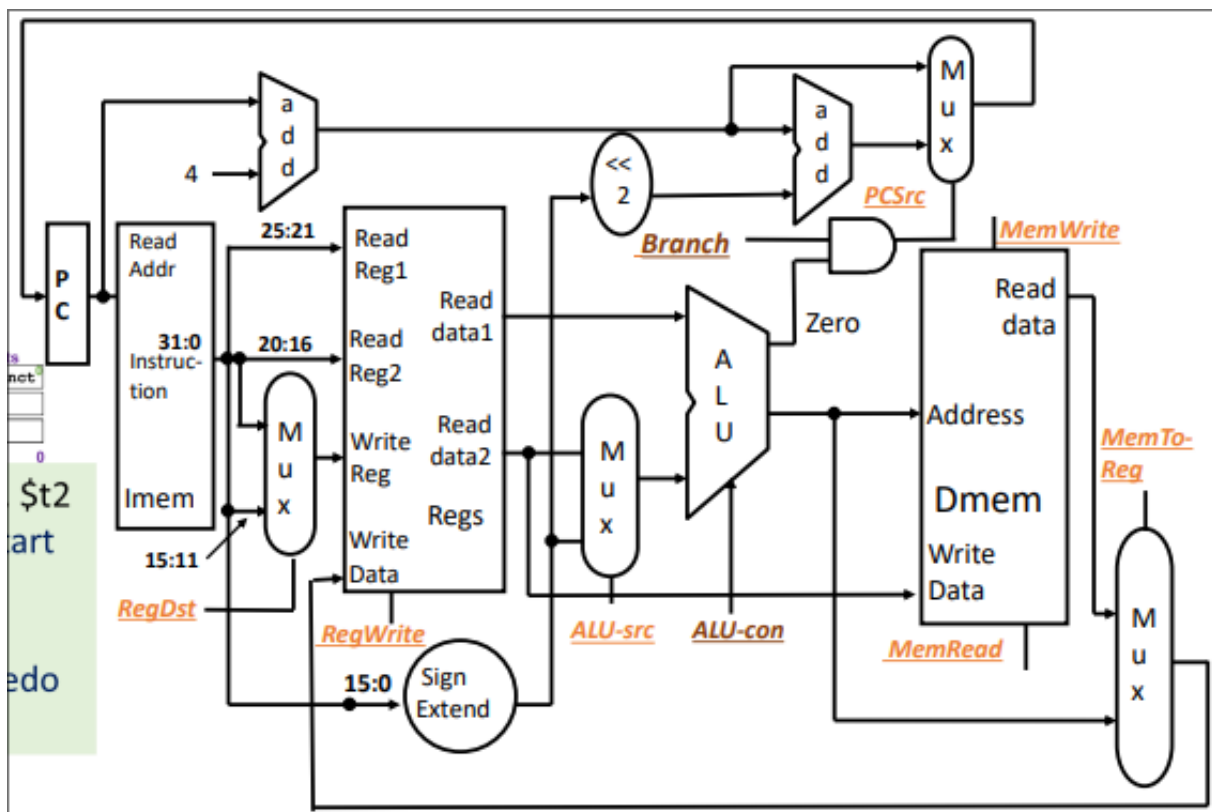
Speedup $\approx 1.23\times$

Thus, the multi-cycle processor is approximately **23% faster** than the single-cycle implementation.

Problem 2

When processor designers consider a possible improvement to the processor datapath, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are starting with a datapath from the given figure, where I-Mem, Add, Mux, ALU, Regs, D-Mem, and Control blocks have latencies of 400 ps, 100 ps, 30 ps, 120 ps, 200 ps, 350 ps, and 100 ps, respectively, and costs of 1000, 30, 10, 100, 200, 2000, and 500, respectively.

Consider the addition of a multiplier to the ALU. This addition will add 300 ps to the latency of the ALU and will add a cost of 600 to the ALU. The result will be 5 % fewer instructions executed since we will no longer need to emulate the MUL instruction.



1. What is the clock cycle time with and without this improvement?
2. What is the speedup achieved by adding this improvement?
3. Compare the cost/performance ration with and without this improvement.

Solution

1. Clock Cycle Time

The clock cycle time of a single-cycle processor is determined by the latency of the longest instruction path. For a **load instruction**, the datapath is:

$$\text{I-Mem} \rightarrow \text{Regs} \rightarrow \text{Mux} \rightarrow \text{ALU} \rightarrow \text{D-Mem} \rightarrow \text{Mux}$$

Why the RegDst MUX is Usually Not in the Critical Path

The **RegDst MUX** selects the destination register. This selection determines which register will receive the result during the write-back stage.

However, the RegDst MUX typically does **not lie on the critical path**. This is because the destination register number is determined **in parallel** with other datapath operations. This is possible due to the fact that the register write operation happens at the very end of the execution of an instruction which means the destination register number is not required until then.

Register latency consideration

If the problem specifies a single register latency value, it is interpreted as the **register file access time for reading**. In this case, the delay is added only once in the critical path calculation. The register write occurs at the clock edge and therefore does not add additional combinational delay.

However, if the problem explicitly provides **separate delays** for register read and register write, then both delays must be included in the clock cycle calculation.

Without Multiplier

$$T_{cycle} = 400 + 200 + 30 + 120 + 350 + 30$$

$$T_{cycle} = 1130 \text{ ps}$$

$T_{cycle}^{old} = 1130 \text{ ps}$

R-type path(add): IMem $\rightarrow$ Reg $\rightarrow$ Mux $\rightarrow$ ALU $\rightarrow$ Mux

$$\begin{aligned}\text{Delay} &= 400 + 200 + 30 + 120 + 30 \\ &= 780 \text{ ps}\end{aligned}$$

I-type path (lw): IMem $\rightarrow$ Reg $\rightarrow$ Mux $\rightarrow$ ALU $\rightarrow$ DMem $\rightarrow$ Mux

$$\begin{aligned}\text{Delay} &= 400 + 200 + 30 + 120 + 350 + 30 \\ &= 1130 \text{ ps}\end{aligned}$$

J-type path(j): IMem $\rightarrow$ Add $\rightarrow$ Mux

$$\begin{aligned}\text{Delay} &= 400 + 100 + 30 \\ &= 530 \text{ ps}\end{aligned}$$

As can be seen, the delay for load word is the highest, so the clock cycle is 1130 ps.

With Multiplier

The multiplier increases the ALU latency by 300 ps.

$$\text{New ALU latency} = 120 + 300 = 420 \text{ ps}$$

$$T_{\text{cycle}} = 400 + 200 + 30 + 420 + 350 + 30$$

$$T_{\text{cycle}} = 1430 \text{ ps}$$

$$\boxed{T_{\text{cycle}}^{\text{new}} = 1430 \text{ ps}}$$

2. Speedup

Execution time is proportional to:

$$\text{Execution Time} = \text{Instruction Count} \times \text{Clock Cycle Time}$$

The improvement reduces the instruction count by 5%.

$$IC_{\text{new}} = 0.95 \times IC_{\text{old}}$$

Speedup:

$$\begin{aligned}\text{Speedup} &= \frac{IC_{\text{old}} \times T_{\text{old}}}{IC_{\text{new}} \times T_{\text{new}}} \\ &= \frac{IC_{\text{old}} \times 1130}{0.95 \times IC_{\text{old}} \times 1430}\end{aligned}$$

$$\begin{aligned}
 &= \frac{1130}{1358.5} \\
 &= 0.83
 \end{aligned}$$

$$\boxed{\text{Speedup} = 0.83}$$

Since the speedup is less than 1, the improvement actually results in a **performance slowdown**.

3. Cost / Performance Comparison

Without Multiplier

$$\begin{aligned}
 \text{Total Cost}_{old} &= 1000 + 2 * 30 + 4 * 10 + 120 + 200 + 2000 + 500 \\
 &= 3900
 \end{aligned}$$

Performance is proportional to:

$$\begin{aligned}
 P_{old} &= \frac{1}{1100} \\
 \frac{\text{Cost}}{\text{Performance}_{old}} &= 3900 \times 1130 \\
 &= 4\,407\,000
 \end{aligned}$$

With Multiplier

Multiplier adds 600 cost.

$$\text{Total Cost}_{new} = 3900 + 600 = 4500$$

Performance:

$$\begin{aligned}
 P_{new} &= \frac{0.95}{1430} \\
 \frac{\text{Cost}}{\text{Performance}_{new}} &= \frac{4500}{0.95/1430} \\
 &= 4460 \times \frac{1430}{0.95} \\
 &\approx 6\,773\,684
 \end{aligned}$$

Conclusion

$$\boxed{\text{Cost/Performance is worse after adding the multiplier}}$$

The multiplier increases hardware cost and clock cycle time, while the instruction count reduction is too small to compensate for the longer cycle. Thus, the modification is **not a good design trade-off**.

Problem 3

For the problems in this exercise, assume that there are no pipeline stalls and that the breakdown of executed instructions is as follows:

add	sub	addi	beq	lw	sw
15%	15%	20%	20%	20%	10%

1. In what fraction of all cycles is the **data memory** accessed?
2. In what fraction of all cycles is the **register file written**? Explain which instructions contribute to this usage.
3. In what fraction of all cycles is the **sign-extend unit** used? What is the output of the sign-extend unit during cycles in which it is not required?

Solution

Instruction distribution:

add	sub	addi	beq	lw	sw
15%	15%	20%	20%	20%	10%

1. Fraction of Cycles Data Memory is Accessed

The data memory is accessed only by **load (lw)** and **store (sw)** instructions.

$$lw = 20\%$$

$$sw = 10\%$$

$$\text{Total} = 20\% + 10\% = 30\%$$

Data memory is accessed in 30% of all cycles

2. Fraction of Cycles Register File is Written

The register file is written by instructions that produce a register result. These include **add**, **sub**, **addi** and **load (lw)**.

$$add = 15\%$$

$$sub = 15\%$$

$$addi = 20\%$$

$$lw = 20\%$$

$$\text{Total} = 15\% + 15\% + 20\% + 20\% = 70\%$$

Register file is written in 70% of all cycles

3. Fraction of Cycles Sign-Extend Unit is Used

The sign-extend unit is used by instructions that contain an immediate field. These include **addi**, **lw**, **sw** and **beq**.

$$addi = 20\%$$

$$lw = 20\%$$

$$sw = 10\%$$

$$beq = 20\%$$

$$\text{Total} = 20\% + 20\% + 10\% + 20\% = 70\%$$

Sign-extend unit is used in 70% of all cycles

Output When Not Used

During cycles when the sign-extend unit is not required (for example with **add** and **sub**), the unit still produces an output because it is combinational logic connected to the instruction bits. However, this output is **ignored by the datapath** since the control signals do not route it to the ALU or other components.

Problem 4

A processor can be implemented using either a **Single-Cycle datapath** or a **Multi-Cycle datapath**. The delays of different hardware components are given in Table 1.

Component	Delay
Instruction Memory	200 ps
Register File Read	100 ps
ALU	150 ps
Data Memory	200 ps
Register Write	100 ps

Table 1: Component delays

Single-Cycle Processor

In a single-cycle processor, the clock cycle must accommodate the **slowest instruction**. Assume the slowest instruction is `lw`, which requires the following components:

Instruction Memory $\rightarrow$ Register Read $\rightarrow$ ALU $\rightarrow$ Data Memory $\rightarrow$ Register Write

Multi-Cycle Processor

In the multi-cycle processor, instruction execution is divided into the following stages:

1. Instruction Fetch
2. Instruction Decode / Register Read
3. Execute
4. Memory Access
5. Write Back

Each stage takes one clock cycle, and the clock period is determined by the **longest stage delay**.

The number of cycles required by each instruction is shown in Table 2.

Instruction Type	Cycles
ALU (add, sub)	4
Load (lw)	5
Store (sw)	4
Branch (beq)	3

Table 2: Cycles required per instruction in multi-cycle processor

Assume the following instruction mix in a program:

Instruction Type	Percentage
ALU operations	70%
Load	10%
Store	10%
Branch	10%

Questions

1. Calculate the **clock cycle time** of the single-cycle processor.
2. Calculate the **execution time for 100 instructions** on the single-cycle processor.
3. Determine the **clock cycle time** of the multi-cycle processor.
4. Compute the **average CPI (Cycles Per Instruction)** for the multi-cycle processor.
5. Calculate the **execution time for 100 instructions** on the multi-cycle processor.
6. Based on your calculations, determine **which processor is faster for this instruction mix**.

Solution

Question 1: Clock Cycle Time

In a single-cycle processor the clock period must accommodate the slowest instruction, which is **lw**. Its critical path traverses every component in sequence:

$$T_{SC} = T_{IM} + T_{RR} + T_{ALU} + T_{DM} + T_{RW} = 200 + 100 + 150 + 200 + 100 = \boxed{750 \text{ ps}}$$

Question 2: Execution Time for 100 Instructions

Every instruction takes exactly one clock cycle in a single-cycle design, so the total execution time is:

$$T_{\text{exec,SC}} = N \times T_{SC} = 100 \times 750 \text{ ps} = \boxed{75,000 \text{ ps} = 75 \text{ ns}}$$

Question 3: Clock Cycle Time

The clock period of a multi-cycle processor equals the delay of its *longest* pipeline stage. The five stages and their delays are:

Stage	Component	Delay
1. Instruction Fetch	Instruction Memory	200 ps
2. Instruction Decode / RR	Register File Read	100 ps
3. Execute	ALU	150 ps
4. Memory Access	Data Memory	200 ps
5. Write Back	Register Write	100 ps

The longest stage delay is $\max(200, 100, 150, 200, 100) = 200$ ps, so:

$$T_{\text{MC}} = \boxed{200 \text{ ps}}$$

Question 4: Average CPI

Using the given instruction mix and cycle counts per instruction:

$$\begin{aligned} \overline{\text{CPI}} &= \sum_i f_i \cdot \text{CPI}_i = (0.70 \times 4) + (0.10 \times 5) + (0.10 \times 4) + (0.10 \times 3) \\ &= 2.80 + 0.50 + 0.40 + 0.30 = \boxed{4.0 \text{ cycles/instruction}} \end{aligned}$$

Question 5: Execution Time for 100 Instructions

$$T_{\text{exec,MC}} = N \times \overline{\text{CPI}} \times T_{\text{MC}} = 100 \times 4.0 \times 200 \text{ ps} = \boxed{80,000 \text{ ps} = 80 \text{ ns}}$$

Question 6: Which Processor is Faster?

Metric	Single-Cycle	Multi-Cycle
Clock cycle time	750 ps	200 ps
CPI	1	4.0
Execution time (100 ins)	75,000 ps (75 ns)	80,000 ps (80 ns)

The speedup of the single-cycle processor over the multi-cycle processor is:

$$\text{Speedup} = \frac{T_{\text{exec,MC}}}{T_{\text{exec,SC}}} = \frac{80 \text{ ns}}{75 \text{ ns}} \approx 1.067$$

Conclusion: For this instruction mix, the **single-cycle processor is faster** by approximately 6.7%. Although the multi-cycle design has a much shorter clock period (200 ps vs. 750 ps), the average of 4.0 cycles per instruction more than offsets that advantage, yielding a higher total execution time of 80 ns compared with 75 ns for the single-cycle processor.

Problem 5

A single-cycle processor and a multi-cycle processor have the following component delays:

- Instruction memory access time = 450 ps
- Data memory access time = 450 ps
- Instruction Decode and Register Read = 180 ps

- Register Write = 180 ps
- ALU delay = 120 ps

Assume that other delays (multiplexers, wires, etc.) are negligible.

A program has the following instruction mix:

- 35% ALU instructions
- 10% load instructions
- 10% store instructions
- 25% branch instructions
- 20% jump instructions

Answer the following questions.

1. For each instruction class, determine which stages are used and compute the total delay. Then determine the clock cycle time of the single-cycle processor.

Instruction Class	Instruction Memory	Decode and Register Read	ALU	Data Memory	Write Back	Total Delay
ALU						
Load						
Store						
Branch						
Jump						

2. Calculate the clock cycle time and the **average CPI (Cycles Per Instruction)** for the multi-cycle processor. Use the following CPI values:

- ALU: 4 cycles
- Load: 5 cycles
- Store: 4 cycles
- Branch: 3 cycles
- Jump: 2 cycles

3. Compute the **speedup** achieved when using the multi-cycle processor compared to the single-cycle processor.

Solution

Component delays:

$$\text{Instruction Memory} = 450 \text{ ps}$$

$$\text{Data Memory} = 450 \text{ ps}$$

$$\text{Decode} + \text{Register Read} = 180 \text{ ps}$$

$$\text{Register Write} = 180 \text{ ps}$$

$$\text{ALU} = 120 \text{ ps}$$

1. Instruction Delays and Single-Cycle Clock Time

Instruction	Instr Mem	Decode / Reg Read	ALU	Data Mem	Write Back	Total Delay
ALU	450	180	120	–	180	930 ps
Load	450	180	120	450	180	1380 ps
Store	450	180	120	450	–	1200 ps
Branch	450	180	120	–	–	750 ps
Jump	450	–	–	–	–	450 ps

The clock cycle time of a single-cycle processor is determined by the **longest instruction delay**.

$$T_{cycle}^{SC} = 1380 \text{ ps}$$

$T_{cycle}^{SC} = 1380 \text{ ps}$

2. Multi-Cycle Processor

Each pipeline stage determines the cycle time.

$$\max(450, 450, 180, 180, 120) = 450 \text{ ps}$$

$T_{cycle}^{MC} = 450 \text{ ps}$

Average CPI

Instruction mix:

$$\begin{aligned}
&\text{ALU} = 35\% \quad (4 \text{ cycles}) \\
&\text{Load} = 10\% \quad (5 \text{ cycles}) \\
&\text{Store} = 10\% \quad (4 \text{ cycles}) \\
&\text{Branch} = 25\% \quad (3 \text{ cycles}) \\
&\text{Jump} = 20\% \quad (2 \text{ cycles})
\end{aligned}$$

$$\begin{aligned}
\text{Average CPI} &= 0.35(4) + 0.10(5) + 0.10(4) + 0.25(3) + 0.20(2) \\
&= 1.4 + 0.5 + 0.4 + 0.75 + 0.4 \\
&= 3.45
\end{aligned}$$

Average $\text{CPI}_{MC} = 3.45$

3. Speedup

Execution time is proportional to:

$$\text{Execution Time} = IC \times CPI \times T_{cycle}$$

Single-cycle processor:

$$CPI_{SC} = 1$$

$$T_{SC} = IC \times 1 \times 1380$$

Multi-cycle processor:

$$T_{MC} = IC \times 3.45 \times 450$$

Speedup:

$$\begin{aligned}
\text{Speedup} &= \frac{IC \times 1380}{IC \times 3.45 \times 450} \\
&= \frac{1380}{1552.5} \\
&= 0.89
\end{aligned}$$

Speedup = 0.89

Thus, in this case the **multi-cycle processor is slower** than the single-cycle processor for this instruction mix.

Problem 6

The component delays of a processor datapath are given in the table below.

Component	Delay (ps)	Cost (units)
Instruction Memory	350	1000
Register File Read	150	200
ALU	200	100
Data Memory	400	2000
Register Write	100	200
Multiplexers (each)	30	10
Control Unit	80	500

Table 3: Component delays and costs

A processor designer proposes **two independent improvements**:

- **Improvement X**: Replace the existing ALU with a faster ALU that reduces the ALU delay by 60 ps but adds a cost of 150 units. This results in 0% change in instruction count.
- **Improvement Y**: Replace the Data Memory with a faster module that reduces the Data Memory delay by 100 ps but adds a cost of 800 units. This change results in 5% fewer instructions due to improved memory-level optimizations.

Assume the **lw** instruction is the critical path:

I-Mem $\rightarrow$ Reg Read $\rightarrow$ ALU $\rightarrow$ D-Mem $\rightarrow$ Reg Write

1. Calculate the **baseline clock cycle time** (without any improvement).
2. Calculate the new clock cycle time and the **speedup** achieved by **Improvement X** alone.
3. Calculate the new clock cycle time and the **speedup** achieved by **Improvement Y** alone.
4. Compare the **cost/performance ratio** for the baseline, Improvement X, and Improvement Y. Which improvement offers the better value?

Solution

Part (a): Baseline Clock Cycle Time

The critical path is determined by the **lw** instruction, which traverses:

$$T_{\text{base}} = T_{\text{IM}} + T_{\text{RR}} + T_{\text{ALU}} + T_{\text{DM}} + T_{\text{RW}} = 350 + 150 + 200 + 400 + 100 = \boxed{1200 \text{ ps}}$$

Part (b): Improvement X (Faster ALU)

Improvement X reduces the ALU delay by 60 ps:

$$T_{\text{ALU},X} = 200 - 60 = 140 \text{ ps}$$

The new clock cycle time becomes:

$$T_X = 350 + 150 + 140 + 400 + 100 = \boxed{1140 \text{ ps}}$$

Since Improvement X results in **0% change in instruction count**, the execution time scales directly with the clock period. The speedup is:

$$\text{Speedup}_X = \frac{\text{Old Execution Time}}{\text{New Execution Time}} = \frac{N \times T_{\text{base}}}{N \times T_X} = \frac{T_{\text{base}}}{T_X} = \frac{1200}{1140} \approx \boxed{1.053}$$

Improvement X yields a speedup of approximately **5.3%**.

Part (c): Improvement Y (Faster Data Memory)

Improvement Y reduces the Data Memory delay by 100 ps:

$$T_{\text{DM},Y} = 400 - 100 = 300 \text{ ps}$$

The new clock cycle time becomes:

$$T_Y = 350 + 150 + 200 + 300 + 100 = \boxed{1100 \text{ ps}}$$

Improvement Y also results in **5% fewer instructions**, so the new instruction count is $N_Y = 0.95 N$. The speedup accounts for both effects:

$$\text{Speedup}_Y = \frac{N \times T_{\text{base}}}{N_Y \times T_Y} = \frac{N \times 1200}{0.95N \times 1100} = \frac{1200}{1045} \approx \boxed{1.148}$$

Improvement Y yields a speedup of approximately **14.8%**.

Part (d): Cost/Performance Ratio

A useful metric for comparing value is the **cost/performance ratio**, defined here as:

$$\text{Cost/Performance} = \frac{\text{Total Cost}}{\text{Performance}}$$

where **Performance** = $1/\text{Execution Time}$, so equivalently:

$$\text{Cost/Performance} = \text{Total Cost} \times \text{Execution Time (relative)}$$

We normalize execution time to the baseline ($= 1$) and compute relative execution time as $1/\text{Speedup}$ for each case.

Baseline Cost

$$C_{\text{base}} = 1000 + 200 + 100 + 2000 + 200 = 3500 \text{ units}$$

(Multiplexers and control unit costs are shared across all designs and cancel in a relative comparison; they are omitted here for clarity.)

Improvement X Cost

$$C_X = C_{\text{base}} + 150 = 3500 + 150 = 3650 \text{ units}$$

Improvement Y Cost

$$C_Y = C_{\text{base}} + 800 = 3500 + 800 = 4300 \text{ units}$$

Relative Execution Times

Taking the baseline execution time as 1:

$$t_{\text{base}} = 1, \quad t_X = \frac{1}{\text{Speedup}_X} = \frac{1}{1.053} \approx 0.950, \quad t_Y = \frac{1}{\text{Speedup}_Y} = \frac{1}{1.148} \approx 0.871$$

Cost/Performance Summary

$$C/P_{\text{base}} = C_{\text{base}} \times t_{\text{base}} = 3500 \times 1.000 = 3500$$

$$C/P_X = C_X \times t_X = 3650 \times 0.950 \approx 3468$$

$$C/P_Y = C_Y \times t_Y = 4300 \times 0.871 \approx 3745$$

Design	Clock (ps)	Speedup	Rel. Exec. Time	Cost (units)	Cost/Performance
Baseline	1200	1.000	1.000	3500	3500
Improvement X	1140	1.053	0.950	3650	3468
Improvement Y	1100	1.148	0.871	4300	3745

Conclusion

- **Best raw performance:** Improvement Y (14.8% faster than baseline).
- **Best cost/performance value: Improvement X**, with a cost/performance ratio of **3468** — lower than both the baseline (3500) and Improvement Y (3745).

Improvement Y, despite delivering greater raw speedup, comes at a significantly higher cost (+800 units vs. +150 units), resulting in a *worse* cost/performance ratio than even the baseline. Improvement X delivers meaningful performance gains at modest additional cost, making it the **better value** overall.